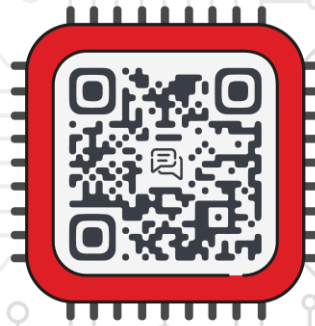




7



11



1320 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Java I/O

عملي مشترك



برمجة ٣

RB Informatics; 4/12/2024

مراجعات.. ➤

File handling

يتم إنشاء كائن يُؤشر على الملف كالتالي:

File f = new File(...);

⬅ بعض الدوال الموجودة في كلاس ال File :

- `FileObject.exists()`: (ال object المنشأ من file) تختبر هذه الدالة فيما إذا كان الملف أو المجلد المرفق بالمسار الممرر للكائن موجوداً أو لا، وهي دالة من نوع Boolean.
- `FileObject.getpath()`: تعيد String يتضمن المسار الممرر للكائن الذي استدعاه.
- `FileObject.createNewFile()`: تقوم بإنشاء ملف جديد.



بعض الكلاسات الموجودة ضمن Java.io:

FileWriter

لكتابة ملف:

```
FileWriter fw = new FileWriter( ... )
```

إما أن نرسل له مسار الملف (String) filePath
أو أن نرسل له File f.

بعض الدوال الموجودة ضمن هذا الكلاس:

`fw.Write(int a)`: تقوم بكتابة محرف واحد على الملف.
`fw.Write(char[] ch)`: تقوم بكتابة مصفوفة المحارف على الملف.
`fw.close()`: تقوم بإغلاق ملف الكتابة و إنهاء جميع عملياته.
`fw.flush()`: تفيد في ضمان نقل المعلومات إلى الملف دون الحاجة لإغلاقه.

FileReader

لقراءة ملف:

```
FileReader fr = new FileReader( ... )
```

إما أن نرسل له مسار الملف (String) filePath
أو أن نرسل له File f.

بعض الدوال الموجودة ضمن هذا الكلاس:

`fr.Read()`: دالة int تقوم بقراءة الملف محرف تلو الآخر إلى حين انتهاء الملف ، إما أن تعيد (-1) أو قيمة المحرف ، و قد ترمي Exception.
`fr.Read(char[] ch)`: دالة int تقوم بقراءة المحارف من المصفوفة المرسلة و تستمر بالقراءة طالما أن المحارف موجودة أو إلى حين انتهاء الملف ، وتعيد (-1) أو عدد المحارف كلها التي تمت قراءتها و ترمي IOException.
`fr.close()`: تقوم بإغلاق الملف القارئ الذي انشأنه وإنهاء جميع العمليات المرتبطة به.

FileInputStream

لكتابة ملف:

```
FileInputStream fis = new FileInputStream ( ... )
```

إما أن نرسل له مسار الملف (String) filePath
أو أن نرسل له File f.

FileOutputStream

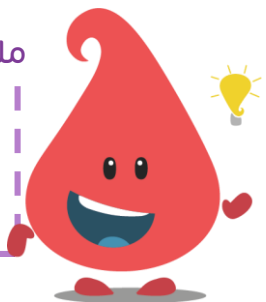
لكتابة ملف:

```
FileOutputStream fos = new FileOutputStream ( ... )
```

إما أن نرسل له مسار الملف (String) filePath
أو أن نرسل له File f.

ملاحظة!!

نستخدم الصئين `FileWriter` و `FileReader` للتعامل مع الملفات النصية فقط ، أما لغير النصية و النصية نستخدم `FileInputStream` و `FileOutputStream`.



ملاحظات مهمة!!

يوجد طريقة أخرى للقراءة من المستخدم تشبه الـ Scanner وهي كالتالي:

```
1 import java.io.*;
2 public class Main {
3     public static void main(String[] args) throws IOException {
4         □
5         InputStreamReader reader = new InputStreamReader(System.in);
6         int text = reader.read();
7         □
8     }}
```

وعند التعامل مع Stream عند انشاء object او استدعاء أحد التوابع فإننا نتعامل مع Checked Exception لذا نكتب throws IOException بجانب تابع الـ main كما يمكننا استخدام الـ try والـ catch ولكن الـ throws متداولة أكثر لأنها اقصر واسهل.

عند انشاء File جديد نقوم بكتابة اسم الملف كما يلي:

```
FileInputStream reader = new FileInputStream( " Path / filename.txt " );
```

المسار الخاص بالملف

اسم الملف

لطباعة عدة محارف نحتاج حلقة while ونكتبها كالتالي:

```
1 int character;
2 while ((character == reader.read()) != -1) {
3     System.out.print((char) character);
4 }
```

حيث سيحصل المتغير character على القيمة -1 عند انتهاء المحارف في الملف

تمارين ➤

1. أكتب الصف الرئيسي SimpleIO يحوي التابع الرئيسي main والذي يقوم بقراءة محارف من دخل المستخدم وطباعتها طالما أن المستخدم لم يُدخل الرقم 0.

```
1 package main3;
2 import java.io.*;
3 public class SimpleIO {
4     public static void main(String args[]) throws IOException {
5         InputStreamReader inp = new InputStreamReader(System.in);
6         System.out.println("Enter characters, " + " and '0' to quit.");
7         char c;
8         do{
9             c = (char) inp.read();
10            System.out.println(c);
11        }while (c != '0');
12    }
```

السطر (2) تضمين مكتبة ال I/O

السطر (3,4) تعريف الصف SimpleIO مع التابع main الرئيسي نقوم بعملية

throws من الـ main للـ IOException

فهذا الاستثناء هو من النوع checked وكان من الممكن أن نضع عملية القراءة ضمن Try block.

السطر (5,7) الطلب من المستخدم إدخال المحارف وقراءة الدخل .

السطر (8,12) طالما أن المستخدم لم يدخل الرقم 0 نقرأ المحرف ونقوم بطباعته



2. أكتب الصف CopyFile يحوي التابع الرئيسي main والذي يقوم بنسخ ملف نصي موجود input.txt إلى ملف output.txt.

```

1 import java.io.*;
2 public class CopyFile {
3     public static void main(String args[]) throws
IOException {
4         FileInputStream in = null;
5         FileOutputStream out = null;
6         try {
7             in = new FileInputStream("input.txt");
8             out = new FileOutputStream("output.txt");
9             int c;
10            while ((c = in.read()) != -1) {
11                out.write(c);
12            }
13        } finally {
14            if (in != null) {
15                in.close();
16            }
17            if (out != null) {
18                out.close();
19            }
20        }
21    }
}

```

السطر (4) انشاء pointer من الصف FileInputStream وتهيئته بـ null.

السطر (5) انشاء pointer من الصف FileOutputStream وتهيئته بـ null.

السطر (6,9) إنشاء الكائنين in للقراءة و out للكتابة، ثم تعريف المتحول c.

السطر (10,11) استخدام حلقة تقرأ كل بايت من الملف باستخدام التابع read() ثم تخزينه في المتغير c، فطالما أنه قام بإعادة لل byte سنقوم بكتابة هذا البايت على الملف out عن طريق التابع write().

السطر (13,18) finally block: إذا كان in لا يساوي null فقم بإغلاق ملف القراءة in، ونقوم بالعملية نفسها على ملف الكتابة out لإغلاقه.

If You Can Dream It,
You Can Do It



3. أكتب الصف `WriteFile` يحوي التابع الرئيسي `main` و الذي يقوم بإنشاء ملف نصي جديد غير موجود مسبقاً وكتابة نص داخله، ثم قراءة النص من هذا الملف وطباعته مع عدد محارفه.

```

1 package main2;
2 import java.io.*;
3 public class WriteFile {
4     public static void main(String[] args) {
5         char [] in = new char[1024];
6         int size = 0;
7         try {
8             File file = new File("lab.txt");
9             FileWriter fw = new FileWriter(file);
10            fw.write(
11                "Programming isn't about what you know;
12                it's about what you can figure out.\n" + "The only
13                way to learn a new programming language is by
14                writing programs.\n");
15            fw.flush();
16            fw.close();
17            FileReader fr = new FileReader(file);
18            size = fr.read(in);
19            System.out.print(size + " characters\n");
20            for(int i = 0; i < size ; i++)
21                System.out.print(in[i]);
22            fr.close();
23        } catch (IOException e) {
24            e.printStackTrace();
25        }
26    }
27 }

```

السطر (2) تضمين مكتبة ال I/O

السطر (3,4) تعريف الصف `WriteFile` مع التابع الرئيسي `main`

السطر (5) تعريف مصفوفة المحارف `in`.

السطر (7) `try block`.

السطر (8) انشاء كائن `file` من `File` و تمرير الملف النصي `lab.txt` له.

السطر (9) انشاء كائن `fw` من ملف الكتابة وتمرير `file` له.

السطر (10,13) كتابة الجملة المطلوبة على الملف عن الطريق التابع ال `write ()` ثم إغلاق `fw`.

السطر (14) انشاء ملف القراءة `fr` وتمرير `file` له.

السطر (15) تخزين عدد محارف الملف ضمن المتحول `size` و ذلك عن الطريق قراءة الملف باستخدام التابع `read()` وتمرير مصفوفة المحارف إليه فالقيمة التي سيعيدها هنا هي عدد محارف الملف.

السطر (16) طباعة محتوى الملف.

السطر (20,21) `catch block` : تعالج ال `Exception` المتوقع حدوثه.

4. اكتب الصف الرئيسي CreateDirectory يحوي التابع الرئيسي main والذي يقوم بإنشاء مجلد اسمه myDir ثم يقوم بإنشاء مسار من المجلدات هي dir1/dir2/dir3.

```
1 package createdirectory;
2 import java.io.*;
3 public class CreateDirectory {
4     public static void main(String args[]){
5         try{
6             String myDirectoy = "myDir";
7             String manyDirectories="dir1/dir2/dir3";
8             boolean success = (new File(myDirectoy)).mkdir();
9             if (success) {
10                 System.out.println("Directory: " + myDirectoy + " created");}
11             success = (new File(manyDirectories)).mkdirs();
12             if (success) {
13                 System.out.println("Directories: " + manyDirectories + " created");}
14         }catch (Exception e){
15             System.out.println("Error: " + e.getMessage());
16         }
17     }
```

السطر (2) تضمين مكتبة ال I/O .

السطر (3,4) تعريف الصف CreateDirectory مع التابع main الرئيسي.

السطر (5) try block .

السطر (6,7) تعريف التحويلات الضرورية لمسارات المجلدات المطلوبة .

السطر (8) تعريف متحول ل boolean باسم success الذي يقوم باستدعاء الدالة mkdir() التي تقوم بإنشاء مجلد ضمن المسار الممرر للملف الذي استدعاه (الكائن) أو بالإسم الممرر . فإن هذا المتحول يعيد قيمة true في نجاح إنشاء الملف .

السطر (12) اختبار فيما إذا تم الإنشاء أم لا .

السطر (14,15) طباعة الاستثناء في حال حدوثه أثناء عمليات الإنشاء

5. أكتب الصف الرئيسي FileDemo يحوي التابع الرئيسي main والذي يقوم بإنشاء مجلد حسب مسار يدخله المستخدم ثم يقوم بإنشاء ملف جديد أيضاً حسب مسار يدخله المستخدم مع التأكد قبل الإنشاء من وجود المجلد أو الملف مسبقاً.

```
1 package filedemo;
2 import java.io.*;
3 import java.util.Scanner;
4 public class FileDemo {
5     public static void main(String args[]) throws IOException {
6         Scanner reader = new Scanner(System.in);
7         boolean success = false;
8         System.out.println("Enter path of directory to create");
9         String dir = reader.nextLine();
10        File directory = new File(dir);
11        if (directory.exists()) {
12            System.out.println("Directory already exists ...");
13        } else {
14            System.out.println("Directory not exists, creating now");
15            success = directory.mkdir();
16            if (success) {
17                System.out.printf("Successfully created new directory : %s\n", dir);
18            } else {
19                System.out.printf("Failed to create new directory: %s\n", dir);
20            }
21            System.out.println("Enter file name to be created ");
22            String filename = reader.nextLine();
23            File f = new File(filename);
24            if (f.exists()) {
25                System.out.println("File already exists");
26            } else {
27                System.out.println("No such file exists, creating now");
28                success = f.createNewFile();
29                if (success) {
30                    System.out.printf("Successfully created new file: %s\n", f);
31                } else {
32                    System.out.printf("Failed to create new file: %s\n", f);
33                }
34            }
35            reader.close();
36        }
37    }
38 }
```



السطر (2,3) تضمين المكاتب الضرورية.

السطر (4,5) تعريف الصف الرئيسي FileDemo والتابع الرئيسي main يرمي Exception.

السطر (6,7) تعريف المتحولات الضرورية.

السطر (8,9) الطلب من المستخدم إدخال مسار المجلد الذي يريد إنشاؤه وقراءة هذا المسار.

السطر (11) تعريف متحول للمجلد بناء على المسار المدخل.

السطر (11,20) في حال وجود المجلد مسبقاً نقوم بطباعة الرسالة تؤكد ذلك ، و في حال عدم وجوده يطبع رسالة تؤكد ذلك و تخبره أنه قيد الإنشاء ، استدعاء التابع mkdir() و إسناد القيمة الناتجة للمتحول success و اختبار قيمة success فإذا كانت هذه القيمة true يظهر للمستخدم رسالة تؤكد ذلك وفي حال الفشل أيضاً يطبع رسالة تؤكد ذلك .

السطر (21,22) الطلب من المستخدم إدخال اسم الملف الذي يريد إنشاؤه وقراءة هذا الاسم

السطر (23) تعريف متحول للملف بناء على الاسم المدخل .

السطر (24,34) في حال وجود الملف مسبقاً نقوم بطباعة الرسالة المناسبة و إلا نقوم بإنشاء الملف وطباعة

HARDWARE:

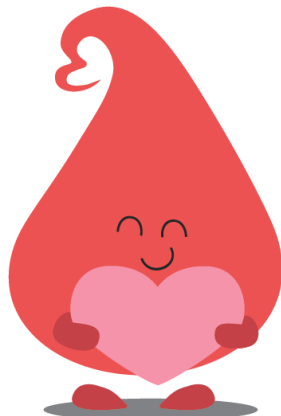
The parts of a computer system that can be kicked.



6. أكتب الصف `FileDetails` الذي يحوي التابع الرئيسي `main` ويقوم بقراءة اسم ملف من دخل المستخدم ثم يقوم بفتح هذا الملف وطباعة محتواه ثم أبحث عن العمليات التالية من مكتبة `I/O` وقم بتنفيذها في برنامجك

1. طباعة حجم الملف .
2. طباعة تاريخ آخر تعديل على الملف .
3. طباعة تاريخ إنشاء الملف .

```
1 package javaio;
2 import java.io.*;
3 import java.nio.file.Files;
4 import java.nio.file.Path;
5 import java.nio.file.attribute.BasicFileAttributes;
6 import java.util.Scanner;
7 public class FileDetails {
8     public static void main(String[] args) throws IOException {
9         Scanner input = new Scanner(System.in);
10        System.out.println("Enter file path");
11        String filepath = input.next();
12        File file = new File(filepath);
13        FileReader f = new FileReader(file);
14        int t;
15        while ((t = f.read()) != -1) {
16            System.out.print((char) t);
17        }
18        System.out.println("");
19        System.out.println("The file length : " + file.length());
20        Path path = file.toPath();
21        BasicFileAttributes att = Files.readAttributes(path, BasicFileAttributes.class);
22        System.out.printf("Last modification time: %s%n", att.lastModifiedTime());
23        System.out.printf("File creation time: %s%n", att.creationTime());
24    }}
```



لن يحمّلك الله مالا تطيقه..

لذلك

أنت دائما تستطيع

السطر (1,6) تضمين المكتبات اللازمةالسطر (7,8) تعريف الصف الرئيسي FileDetails والتابع الرئيسي main يرمي Exception.السطر (10,11) إظهار رسالة للمستخدم تطاب منه إدخال الـ file path (مسار الفايل) ثم ادخاله .السطر (12) إنشاء object من الـ File وتمرير filepath كباراميتز للباني .السطر (13) إنشاء object من الـ FileReader وتمرير الـ Filepath له كباراميتز للباني .السطر (14) تعريف المتحول t من النوع int .السطر (15، 17) حلقة while تقوم بطباعة t كـ char طالما أن r.read() غير فارغة .السطر (19) طباعة قيمة الـ f.length سيتم طباعة القيمة بالـ MB إذا أردنا القيمة بالـ Bits نضرب بـ 8أما إذا أردناها بالـ KB نقسم على 1024.السطر (20) ثم ننشأ كائن من الـ Path الذي يستدعي file.toPath()، بحيث يمكننا الحصول على مسار الملف من خلال الدالة toPath() التي نستدعيها من الملف الذي نريد مساره ، فتعيد مسار كامل التي قامت بتخزينه في object من الـ path.السطر (21) إنشاء obj من BasicFileAttributes لأنه في java لا يمكننا الوصول إلى خواص كالتاريخ ، تاريخ الإنشاء وآخر تعديل وآخر وصول ... إلا عن طريق الواجهة BasicFileAttributes أو أحد أبنائها .فبإنشاء كائن منه نستطيع أن نخزن فيه جميع خواص الملف ثم نصل عن طريقه إلى الخواص التي نريدها. الدالة readAttributes() الموجودة في الكلاس Files نقوم بتمرير مسار الملف الذي نريد الحصول علىخصائصه كباراميتز لها هي والواجهة الذي ستحوي على خصائص الملف ، وهو BasicFileAttributes فتقوم هذه الدالة بإرجاع خصائص هذا الملف بعدما قامت بتخزينها بكائن يطبق BasicFileAttributes.السطر (22,23) طباعة تاريخ إنشاء الملف وتاريخ آخر تعديل على الملف